



254383 Algorithm Design and Analysis

อ.พิเศษพงษ์ สุธาพันธ์
ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ





Week 3

Brute Force and Exhaustive Search

อ.พิเศษพงษ์ สุธาพันธ์
ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ





Brute force

วัตถุประสงค์

1. เข้าใจแนวคิดการทำงานแบบ Brute force
2. สามารถนำไปประยุกต์ใช้งานได้



Brute force

เป็นแนวคิดทำงานแก้ปัญหาแบบตรงๆ
บนพื้นฐานของปัญหา ไม่ต้องคิด
ซับซ้อน



Exhaustive Search

ค้นหาทุกทางเพื่อหาคำตอบ



Bubble Sort

แนวคิด

$A_0, \dots, A_j \overset{?}{\leftrightarrow} A_{j+1}, \dots, A_{n-i-1} \mid A_{n-i} \leq \dots \leq A_{n-1}$
in their final positions

วิดีโอ

<https://www.youtube.com/watch?v=nmhjrl-aW5o>

<https://www.youtube.com/watch?v=TZRWRjq2CAg>



Bubble Sort

ALGORITHM *BubbleSort*($A[0..n - 1]$)

//Sorts a given array by bubble sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow 0$ **to** $n - 2 - i$ **do**

if $A[j + 1] < A[j]$ swap $A[j]$ and $A[j + 1]$



Bubble Sort

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1] \\ &= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2). \end{aligned}$$

$$S_{worst}(n) = C(n) = \frac{(n-1)n}{2} \in \Theta(n^2).$$



Selection Sort

แนวคิด

$A_0 \leq A_1 \leq \dots \leq A_{i-1}$ | $A_i, \dots, A_{min}, \dots, A_{n-1}$
in their final positions the last $n - i$ elements

วิดีโอ

<https://www.youtube.com/watch?v=xWBP4IzkoyM>



Selection Sort

ALGORITHM *SelectionSort*($A[0..n - 1]$)

//Sorts a given array by selection sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

$min \leftarrow i$

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[j] < A[min]$ $min \leftarrow j$

 swap $A[i]$ and $A[min]$



Selection Sort

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i)$$

$$\Theta(n^2)$$



3.2

Sequential Search and Brute-Force String Matching



Sequential Search

วิดีโอ

<https://www.youtube.com/watch?v=x1d1b6Rb--E>



Sequential Search

ALGORITHM *SequentialSearch2*($A[0..n]$, K)

//Implements sequential search with a search key as a sentinel

//Input: An array A of n elements and a search key K

//Output: The index of the first element in $A[0..n - 1]$ whose value is

// equal to K or -1 if no such element is found

$A[n] \leftarrow K$

$i \leftarrow 0$

while $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1



Brute-Force String Matching

Text คือ ข้อความทั้งหมดที่จะทำการ
ค้นหาที่มีความยาวเท่ากับ n

Pattern คือ ข้อความที่ต้องการ
ค้นหาใน Text ซึ่งมีขนาดเท่ากับ m

วิดีโอ

<https://www.youtube.com/watch?v=vtnpzDPgaU0>



Brute-Force String Matching

การค้น pattern ใน text จะเจอกี่ต่อเมื่อ

$$t_i = p_0, \dots, t_{i+j} = p_j, \dots, t_{i+m-1} = p_{m-1}$$

t_0	$\dots$	t_i	$\dots$	t_{i+j}	$\dots$	t_{i+m-1}	$\dots$	t_{n-1}	text T
		$\updownarrow$		$\updownarrow$		$\updownarrow$			
		p_0	$\dots$	p_j	$\dots$	p_{m-1}			pattern P



Brute-Force String Matching

ALGORITHM *BruteForceStringMatch*($T[0..n-1]$, $P[0..m-1]$)

//Implements brute-force string matching

//Input: An array $T[0..n-1]$ of n characters representing a text and

// an array $P[0..m-1]$ of m characters representing a pattern

//Output: The index of the first character in the text that starts a

// matching substring or -1 if the search is unsuccessful

for $i \leftarrow 0$ **to** $n - m$ **do**

$j \leftarrow 0$

while $j < m$ **and** $P[j] = T[i + j]$ **do**

$j \leftarrow j + 1$

if $j = m$ **return** i

return -1



Brute-Force String Matching

$O(nm)$



Text : 'a new Doctor Strange movie'

Pattern : 'new'

Pattern : 'cle'



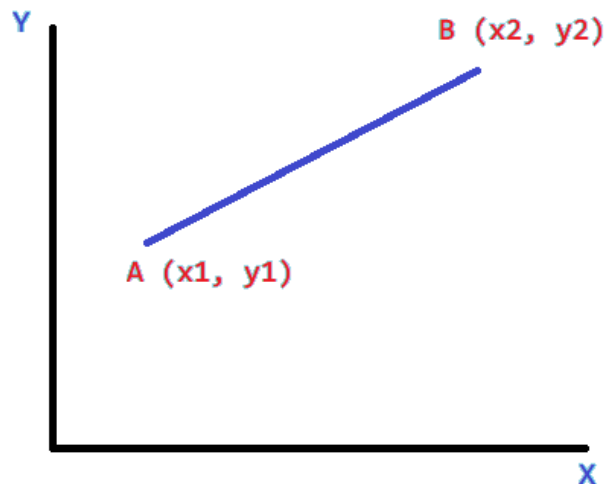
3.3 Closest-Pair and Convex-Hull Problems by Brute Force



Closest-Pair Problem

ระยะห่างระหว่างจุด 2 จุด

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$





Closest-Pair Problem

1

2

3

4



Closest-Pair Problem

ALGORITHM *BruteForceClosestPair(P)*

//Finds distance between two closest points in the plane by brute force

//Input: A list P of n ($n \geq 2$) points $p_1(x_1, y_1), \dots, p_n(x_n, y_n)$

//Output: The distance between the closest pair of points

$d \leftarrow \infty$

for $i \leftarrow 1$ **to** $n - 1$ **do**

for $j \leftarrow i + 1$ **to** n **do**

$d \leftarrow \min(d, \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2))$ //sqrt is square root

return d

หาคู่พิกัดที่มีระยะทางสั้น
ที่สุด



Closest-Pair Problem

$$\begin{aligned} C(n) &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2 = 2 \sum_{i=1}^{n-1} (n - i) \\ &= 2[(n - 1) + (n - 2) + \cdots + 1] = (n - 1)n \in \Theta(n^2) \end{aligned}$$

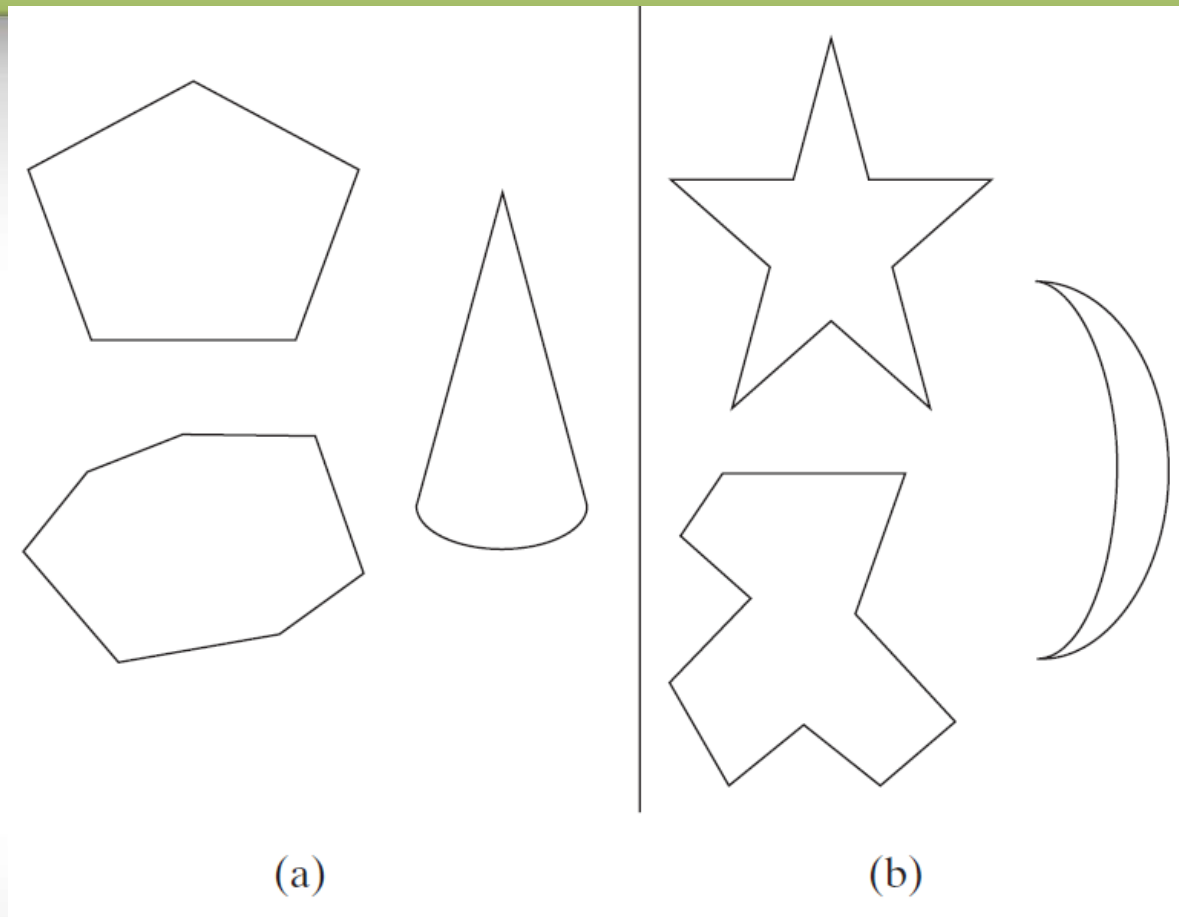


Convex-Hull Problem

DEFINITION ชุดของจุด (จำกัด หรือ ไม่จำกัด) ที่อยู่ในระนาบ จะเรียกว่า *convex* ก็ต่อเมื่อ ทุกสองจุดใดๆ ทั้งจุด p และจุด q อยู่ในกลุ่ม โดยเส้นที่เชื่อมโยงจุดใดๆ ไม่อยู่นอกขอบเขตของเส้นเชื่อมรอบนอก



Convex-Hull Problem

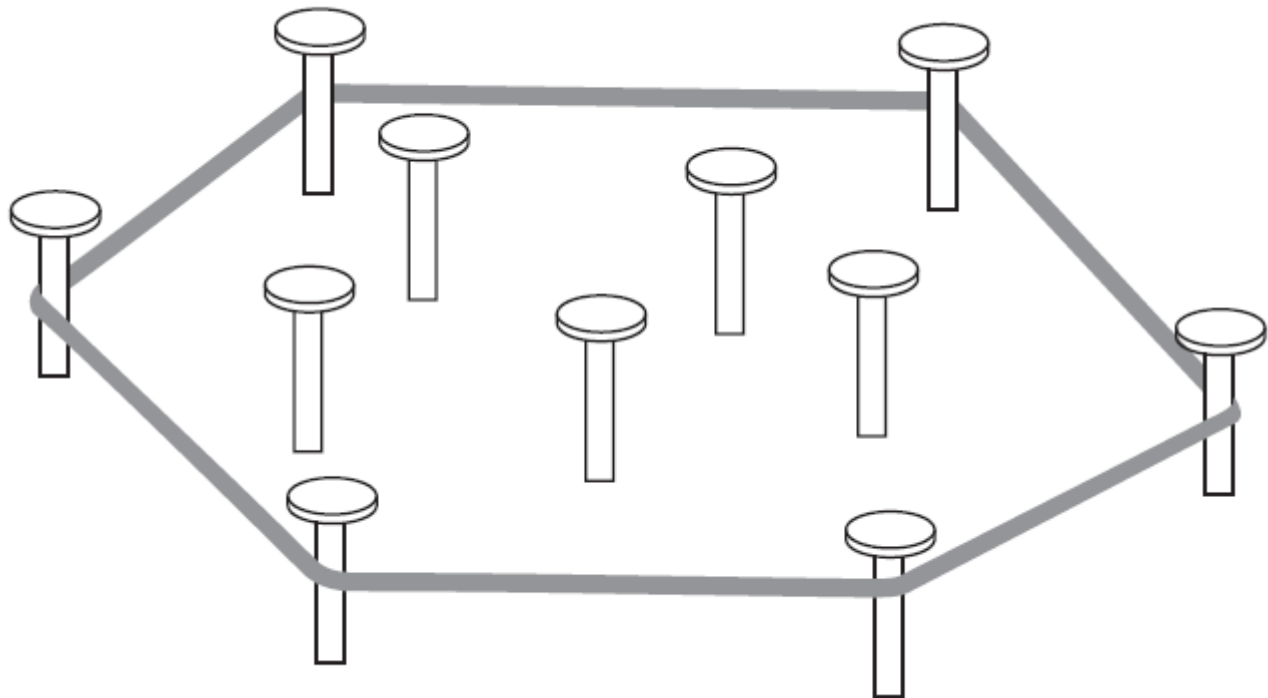


(a) Convex sets. (b) Sets that are not convex.



Convex-Hull Problem

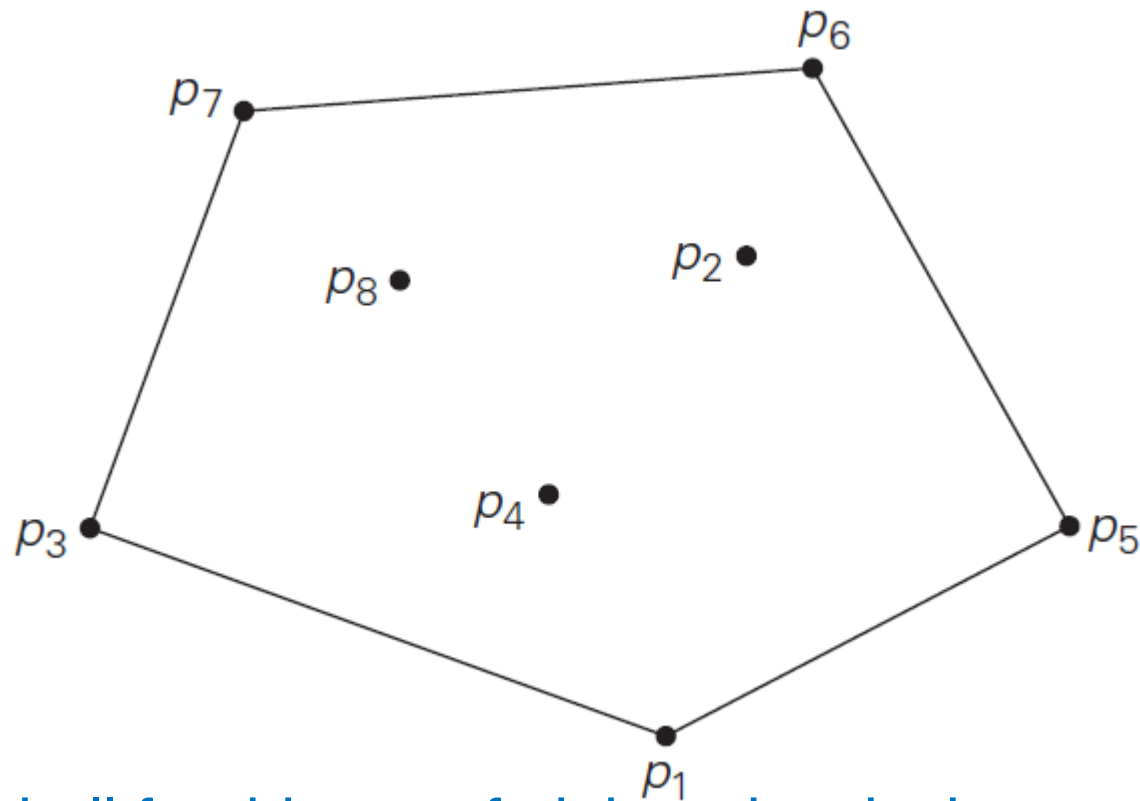
DEFINITION : convex hull ของเซต S คือ เซตของ convex ที่มีขนาดเล็กที่สุดใน S



Rubber-band interpretation of the convex hull



Convex-Hull Problem



The convex hull for this set of eight points is the convex polygon with vertices at p_1 , p_5 , p_6 , p_7 , and p_3 .



Convex-Hull Problem

ประสิทธิภาพอยู่ใน $O(n^3)$

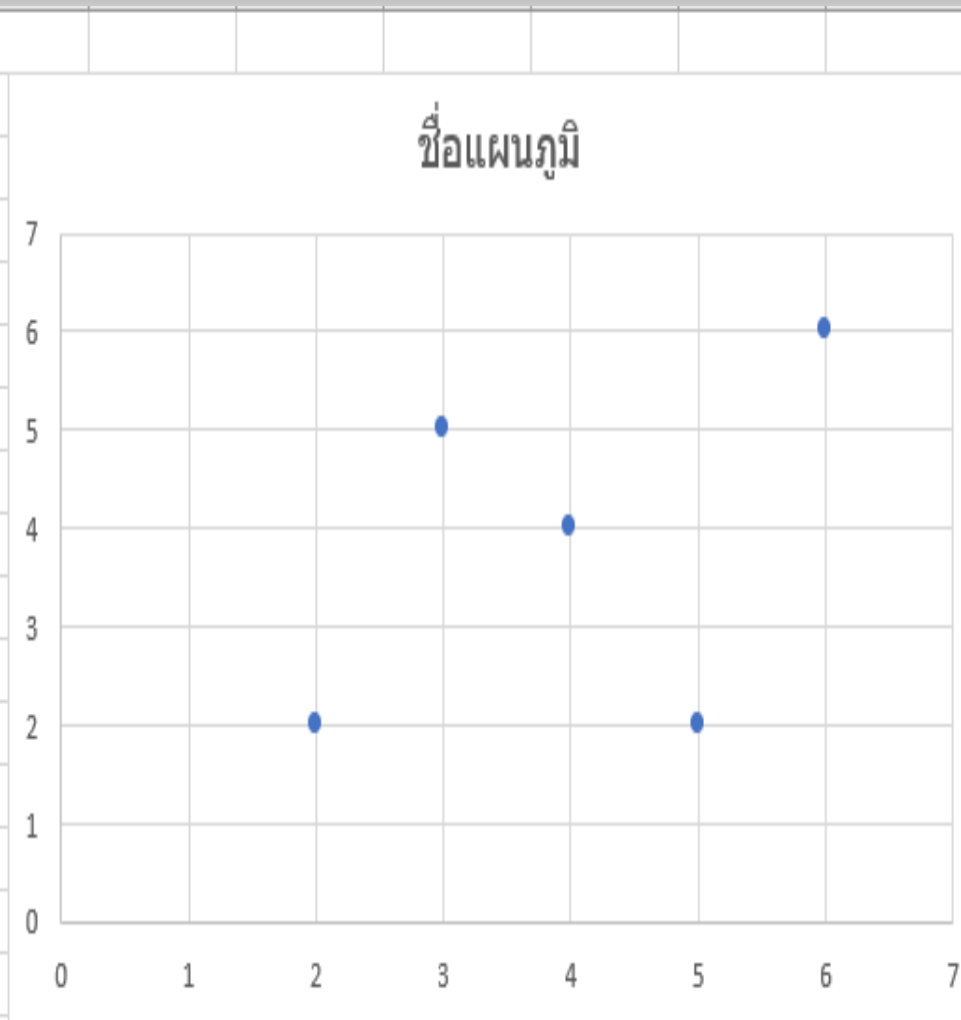
Two points $(x_1, y_1), (x_2, y_2)$ make the line $ax + by = c$
where $a = y_2 - y_1$, $b = x_1 - x_2$, and $c = x_1y_2 - y_1x_2$

And divides the plane by $ax + by - c < 0$ and $ax + by - c > 0$
So we need to only check $ax + by - c$ for the rest of the points



Convex-Hull Problem

y	2	3	4	5	6
x	2	5	4	2	6
$m = (y_2 - y_1) / (x_2 - x_1)$	0.333333	1	#DIV/0!	1	
$a = y_2 - y_1$		1	2	3	4
$b = x_1 - x_2$		-3	-2	0	-4
$c = x_1 y_2 - y_1 x_2$		-4	0	6	0
$(2,2)(6,6)$	$4x + -4y = 0$				
$(4,4)$	$4(4) + -4(4) = 0$				
$(3,5)$	$4(3) + -4(5) < 0$				
$(5,2)$	$4(5) + -4(2) > 0$				





3.4

Exhaustive Search



Exhaustive search

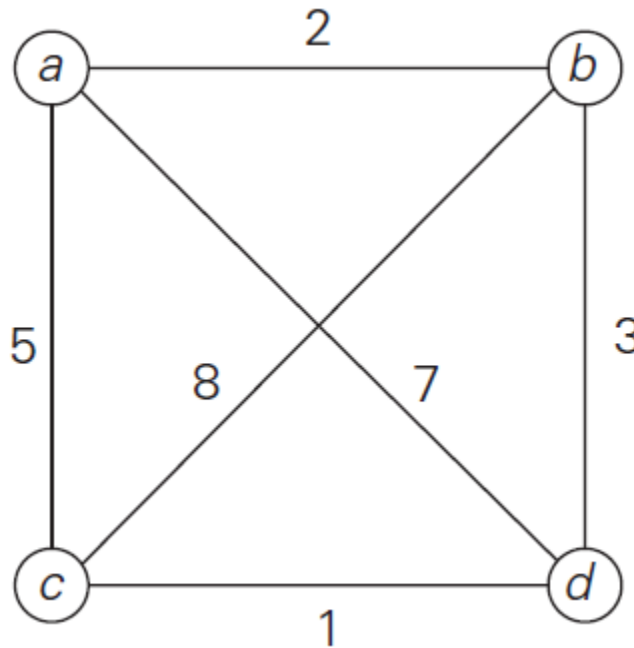
เป็นแนวพื้นฐานของ brute-force ที่
เกี่ยวข้องกับการจัดกลุ่ม ซึ่งต้องมีการ
ตรวจสอบทุกกรณีที่เป็นไปได้

ตัวอย่างคือปัญหาต่อไปนี้



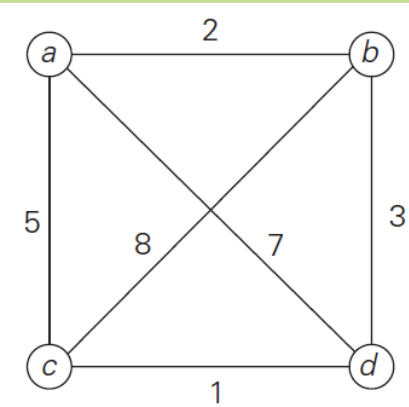
Traveling Salesman Problem

การเดินทางไปยังทุกจุดและกลับมายังจุดเริ่มต้นโดยใช้ระยะทางที่น้อยที่สุด





Traveling Salesman Pro



<u>Tour</u>	<u>Length</u>	
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$l = 2 + 8 + 1 + 7 = 18$	
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$l = 2 + 3 + 1 + 5 = 11$	optimal
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$l = 5 + 8 + 3 + 7 = 23$	
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$l = 5 + 1 + 3 + 2 = 11$	optimal
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$l = 7 + 3 + 8 + 5 = 23$	
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$l = 7 + 1 + 8 + 2 = 18$	

Solution to a small instance of the traveling salesman problem by exhaustive search.



Traveling Salesman Problem

กรณีนี้ เป็นปัญหา permutation (การเรียงสับเปลี่ยน)
ซึ่งจะมีวิธีการคิดทางเดินได้ดังนี้

มี 4 จุดที่ต้องเดินทางไป และกลับวนมายังจุดเริ่มต้น

กำหนดจุดเริ่มต้น

จุดที่จะเดินต่อไป

จุดที่จะเดินต่อไป

จุดที่จะเดินต่อไป

วนกลับที่เดิม

A

(B,C,D)

(C,D)

(D)

(A)

1

3

2

1

1

เส้นทางทั้งหมดที่เป็นไปได้ คือ $1*3*2*1*1 = 6$ เส้นทาง



Traveling Salesman Problem

Tour

Length

$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$

$$l = 2 + 8 + 1 + 7 = 18$$

$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$

$$l = 2 + 3 + 1 + 5 = 11 \quad \text{optimal}$$

$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$

$$l = 5 + 8 + 3 + 7 = 23$$

$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$

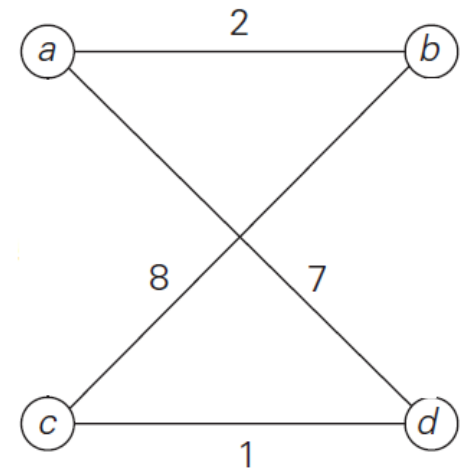
$$l = 5 + 1 + 3 + 2 = 11 \quad \text{optimal}$$

$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$

$$l = 7 + 3 + 8 + 5 = 23$$

$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$

$$l = 7 + 1 + 8 + 2 = 18$$





Traveling Salesman Problem

เส้นทางทั้งหมดที่เป็นไปได้ คือ $1*3*2*1*1 = 6$ เส้นทาง

เพราะฉะนั้นประสิทธิภาพจะอยู่ใน $O(n!)$

<http://10.31.26.28/~phisetphong/algorithm/travelingselesmanbruteforce.html>

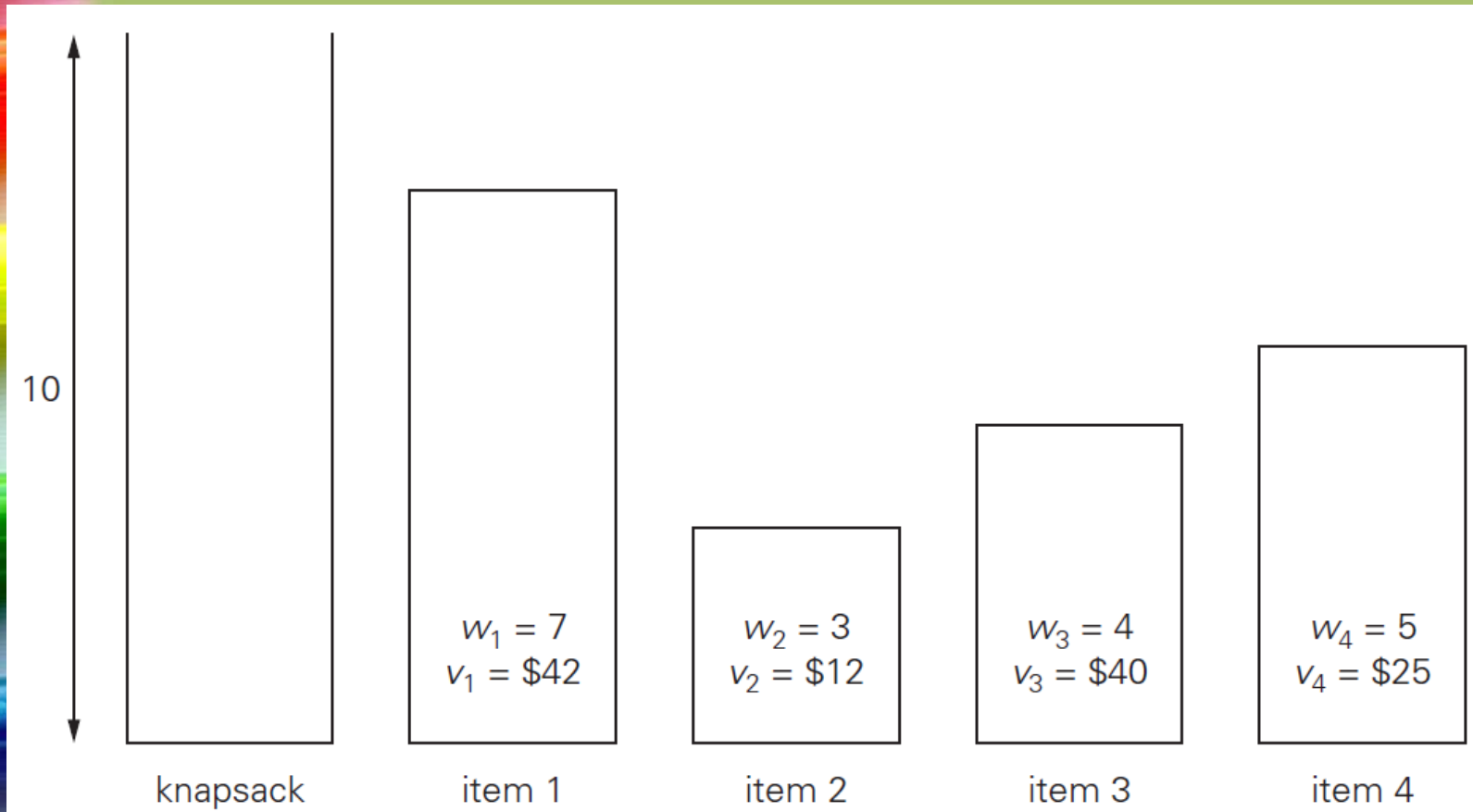


Knapsack Problem

มีของ n สิ่ง แต่ละอันมีน้ำหนักเป็น $w_1, w_2, \dots, w_n$ และมีมูลค่าเป็น $v_1, v_2, \dots, v_n$ โดยคุณมีกระสอบที่รับน้ำหนักได้ W คุณจะเก็บสิ่งใดใส่กระสอบบ้างเพื่อให้ได้มูลค่ามากที่สุด



Knapsack Problem





Knapsack Problem

Subset	Total weight	Total value
$\emptyset$	0	\$ 0
{1}	7	\$42
{2}	3	\$12
{3}	4	\$40
{4}	5	\$25
{1, 2}	10	\$54
{1, 3}	11	not feasible
{1, 4}	12	not feasible
{2, 3}	7	\$52
{2, 4}	8	\$37
{3, 4}	9	\$65
{1, 2, 3}	14	not feasible
{1, 2, 4}	15	not feasible
{1, 3, 4}	16	not feasible
{2, 3, 4}	12	not feasible
{1, 2, 3, 4}	19	not feasible



Knapsack Problem

ของ 1 สิ่งมีวิธีการเลือกอยู่ 2 วิธี คือ เลือก กับ ไม่เลือก

เพราะฉะนั้นถ้าของ n สิ่ง จะมีวิธีเลือก 2^n วิธี

ดังนั้น กรณีนี้จะอยู่ใน $\Omega(2^n)$

<http://10.31.26.28/~phisetphong/algorithm/knapsackbruteforce.html>



Assignment Problem

ต.ย. มีคน n คน มีงาน n งาน โดยให้งาน
คนละหนึ่งงาน โดยให้ค่าจ้างรวมน้อยที่สุด

	Job 1	Job 2	Job 3	Job 4
Person 1	9	2	7	8
Person 2	6	4	3	7
Person 3	5	8	1	8
Person 4	7	6	9	4



Assignment Problem

ทางเลือกในการกำหนดงาน คิดจาก

คนแรก

คนที่สอง

คนที่สาม

คนที่สี่

งานเหลือ

งานเหลือ

งานเหลือ

งานเหลือ

4

3

2

1



Assignment Problem

ทางเลือกในการกำหนดงาน

$$4 * 3 * 2 * 1 = 4!$$

ดังนั้นจึงมีเท่ากับ 24 ทางเลือก



Assignment Problem

$$C = \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix}$$

$\langle 1, 2, 3, 4 \rangle$

$$\text{cost} = 9 + 4 + 1 + 4 = 18$$

$\langle 1, 2, 4, 3 \rangle$

$$\text{cost} = 9 + 4 + 8 + 9 = 30$$

$\langle 1, 3, 2, 4 \rangle$

$$\text{cost} = 9 + 3 + 8 + 4 = 24$$

$\langle 1, 3, 4, 2 \rangle$

$$\text{cost} = 9 + 3 + 8 + 6 = 26$$

$\langle 1, 4, 2, 3 \rangle$

$$\text{cost} = 9 + 7 + 8 + 9 = 33$$

$\langle 1, 4, 3, 2 \rangle$

$$\text{cost} = 9 + 7 + 1 + 6 = 23$$

etc.

First few iterations of solving a small instance of the assignment problem by exhaustive search.



3.5

Depth-First Search and Breadth-First Search



Depth-First Search and Breadth-First Search

การท่องเข้าไปในกราฟ โดยเยือนทุกโหนด โดย
เยือนแต่ละโหนดเพียง 1 ครั้งเท่านั้น ซึ่งจะมี
วิธีการทำงานอยู่ 2 แนวทางหลักๆคือ เทคนิค
การท่องในแนวลึก (Depth-first traversal) และ
เทคนิคการท่องในแนวกว้าง (Breadth-first
traversal)

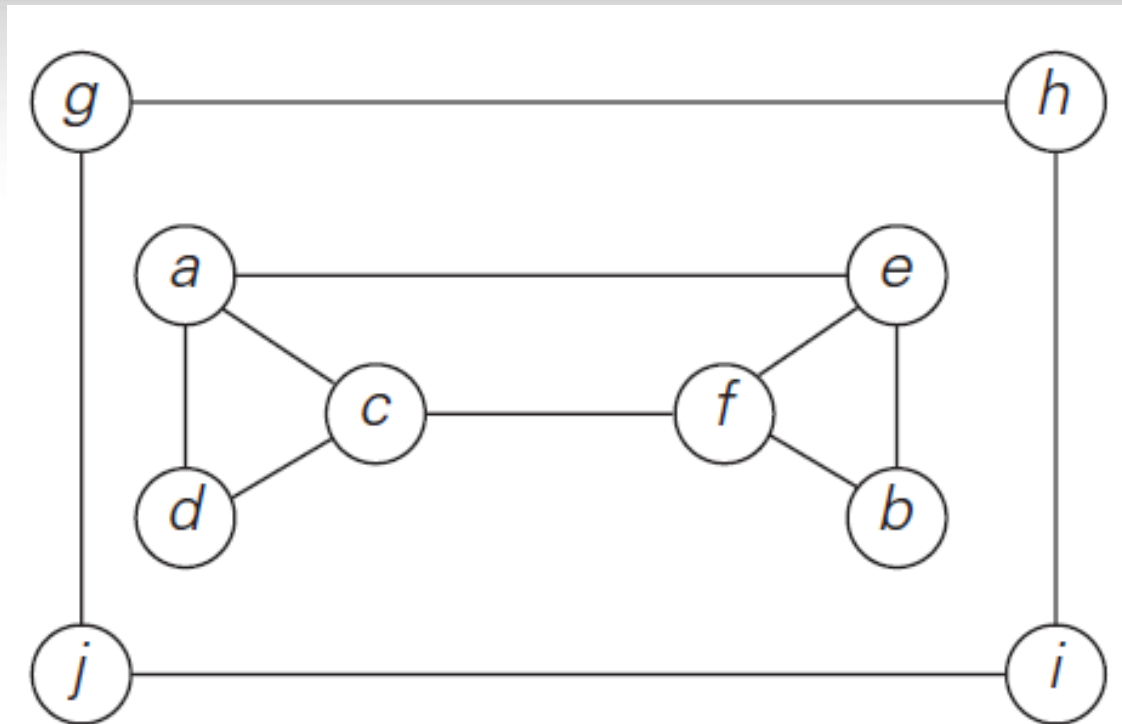


Depth-First Search(DFS)

ไล่ไปตามโหนดที่เชื่อมต่อกัน เมื่อสิ้นสุด
แล้ว ค่อยมาเริ่มจากตัวท้ายสุดที่สามารถ
ไปยังเส้นทางอื่นได้



Depth-First Search(DFS)



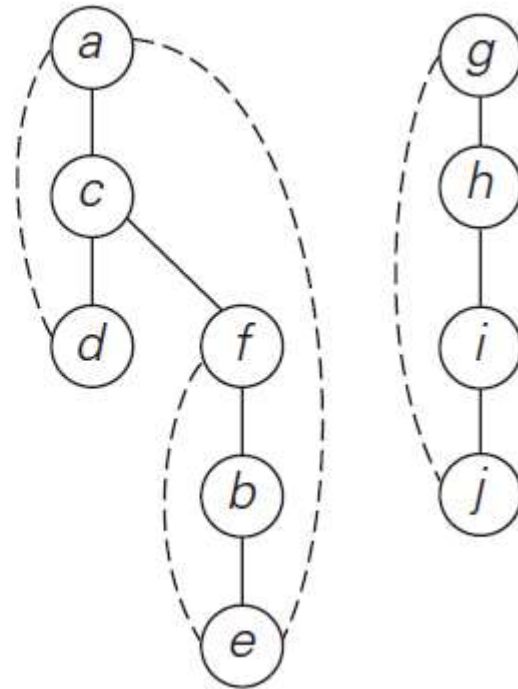
(a) Graph.



Depth-First Search(DFS)

$e_{6,2}$
 $b_{5,3}$ $j_{10,7}$
 $d_{3,1}$ $f_{4,4}$ $i_{9,8}$
 $c_{2,5}$ $h_{8,9}$
 $a_{1,6}$ $g_{7,10}$

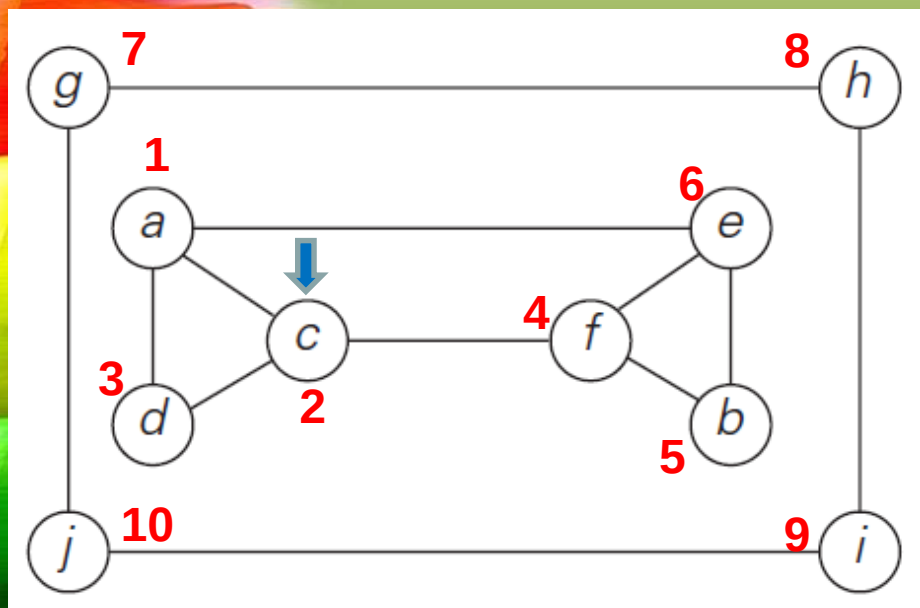
(b) Traversal's stack



(c) DFS forest with the tree and back edges



Depth-First Search(DFS)

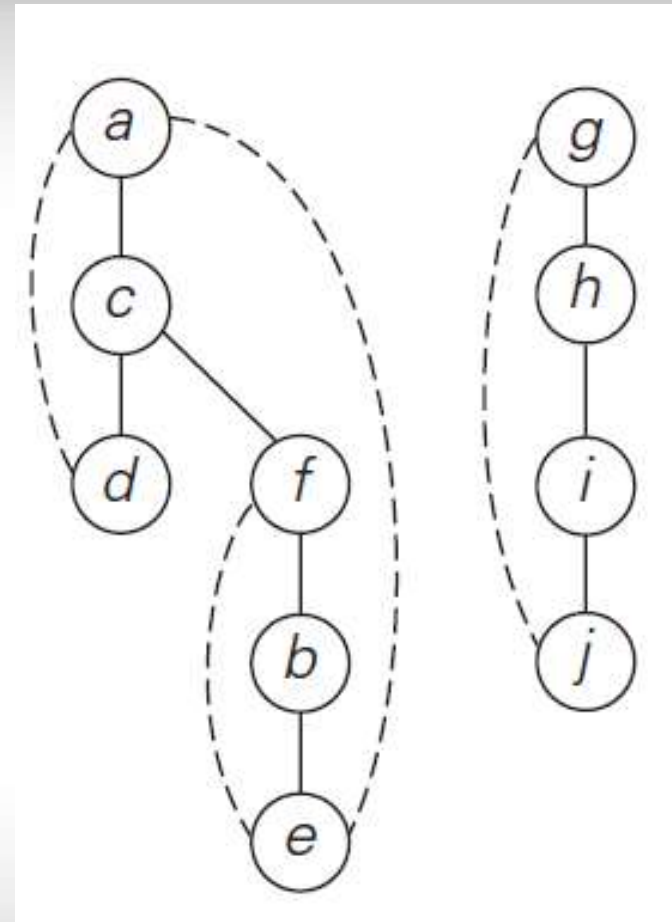
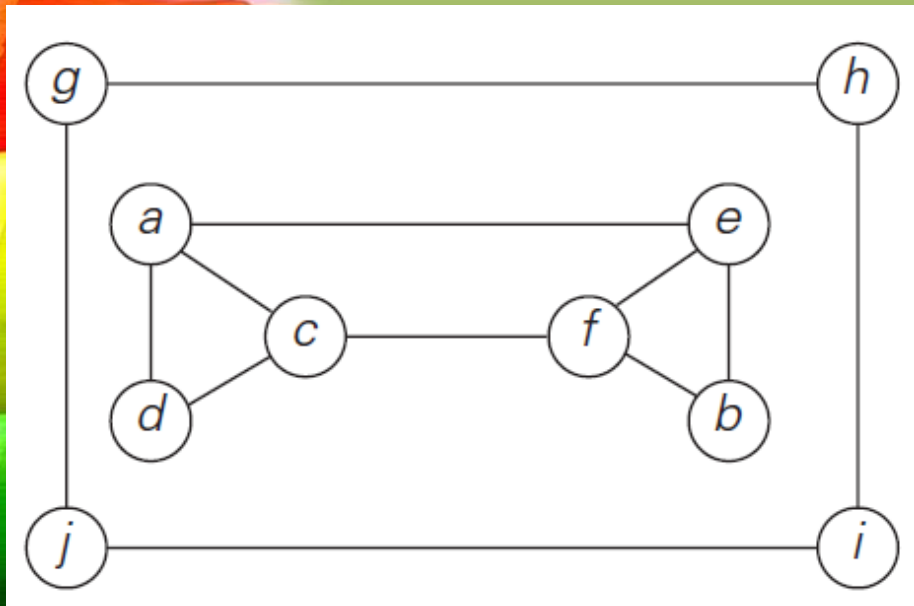


DFS traversal sequence (node, discovery time):

- $\rightarrow e_6, 2$
- $\rightarrow b_5, 3$
- $\rightarrow d_3, 1$
- $\rightarrow f_4, 4$
- $\rightarrow i_9, 8$
- $\rightarrow h_8, 9$
- $\rightarrow g_7, 10$
- $\rightarrow c_2, 5$
- $\rightarrow a_1, 6$
- $\rightarrow j_{10}, 7$



Depth-First Search(DFS)





Depth-First Search(DFS)

ALGORITHM $DFS(G)$

//Implements a depth-first search traversal of a given graph

//Input: Graph $G = \langle V, E \rangle$

//Output: Graph G with its vertices marked with consecutive integers

// in the order they are first encountered by the DFS traversal

mark each vertex in V with 0 as a mark of being “unvisited”

$count \leftarrow 0$

for each vertex v in V **do**

if v is marked with 0

$dfs(v)$



Depth-First Search(DFS)

dfs(v)

//visits recursively all the unvisited vertices connected to vertex v
//by a path and numbers them in the order they are encountered
//via global variable *count*

$count \leftarrow count + 1$; mark v with *count*

for each vertex w in V adjacent to v **do**

if w is marked with 0

dfs(w)



Depth-First Search(DFS)

ALGORITHM $DFS(G)$

mark each vertex in V with 0 as a mark of being “unvisited”

$count \leftarrow 0$

for each vertex v in V **do**

if v is marked with 0

$dfs(v)$

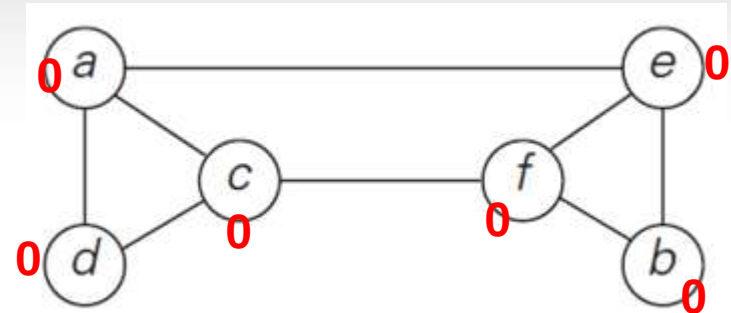
$dfs(v)$

$count \leftarrow count + 1$; mark v with $count$

for each vertex w in V adjacent to v **do**

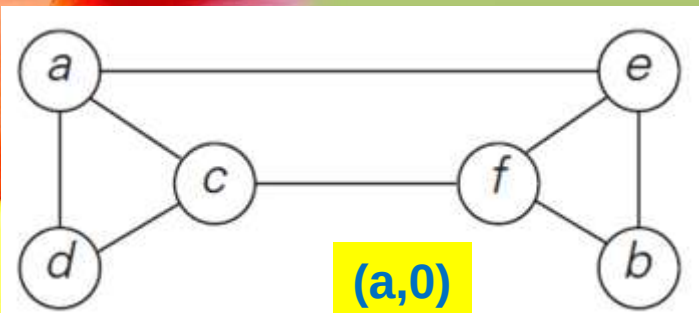
if w is marked with 0

$dfs(w)$





Depth-First Search(DFS)



(a,0)

6

(a,1)



(c,d)

(c,0)

5

(c,2)



(d,f)

(d,0)

1

(d,3)



{}

(f,0)

4

(f,4)



(b,e)

(b,0)

3

(b,5)



(e)

(e,0)

2

(e,6)



{}

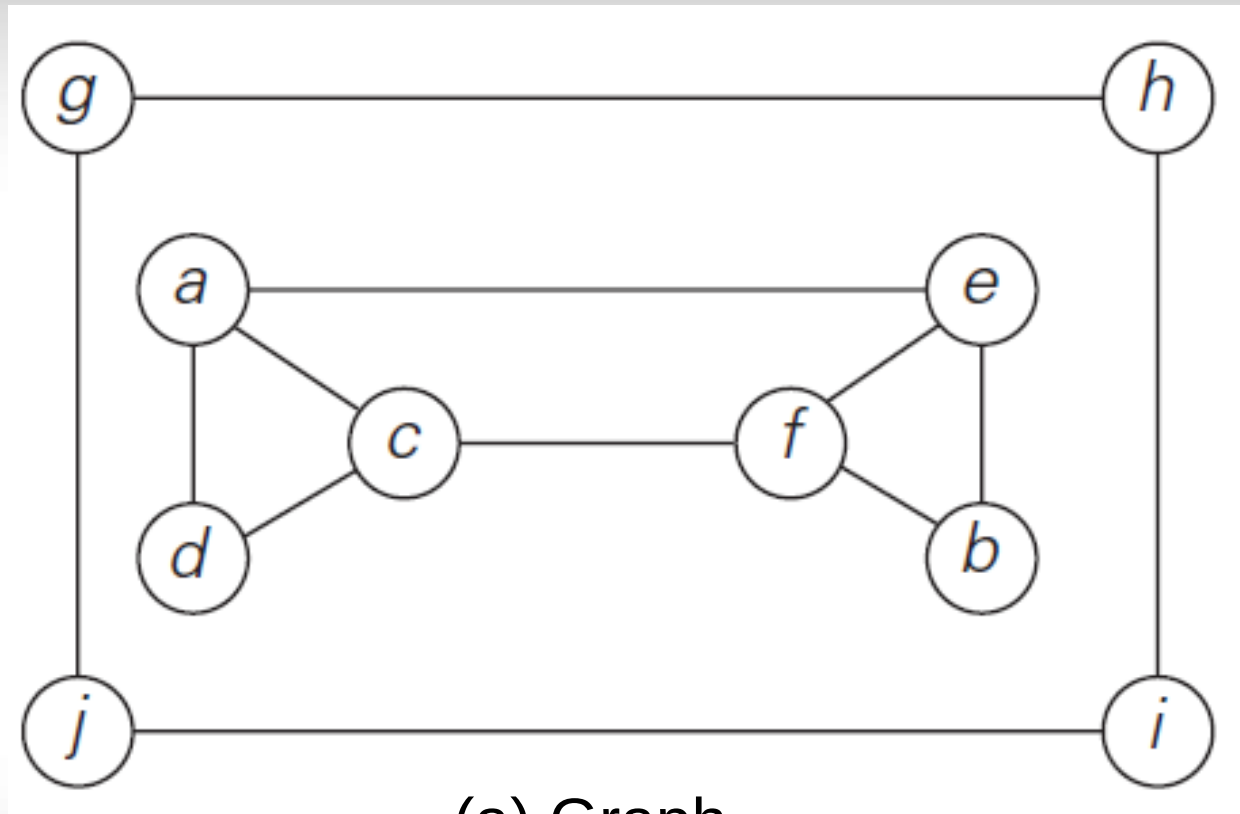


Breadth-First Search(DFS)

เริ่มจากโหนดแม่โหนดหนึ่ง แล้วเรียงไป
ตามโหนดลูกของมัน จากนั้นก็เรียงไป
ตามโหนดหลานของโหนดลูก ไปเรื่อยๆ



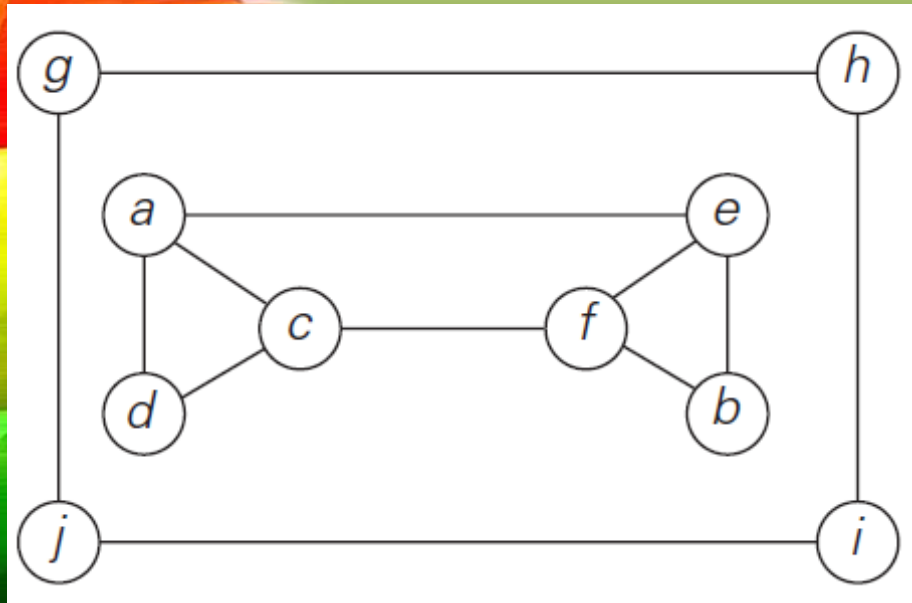
Breadth-First Search(DFS)



(a) Graph.



Breadth-First Search(BFS)

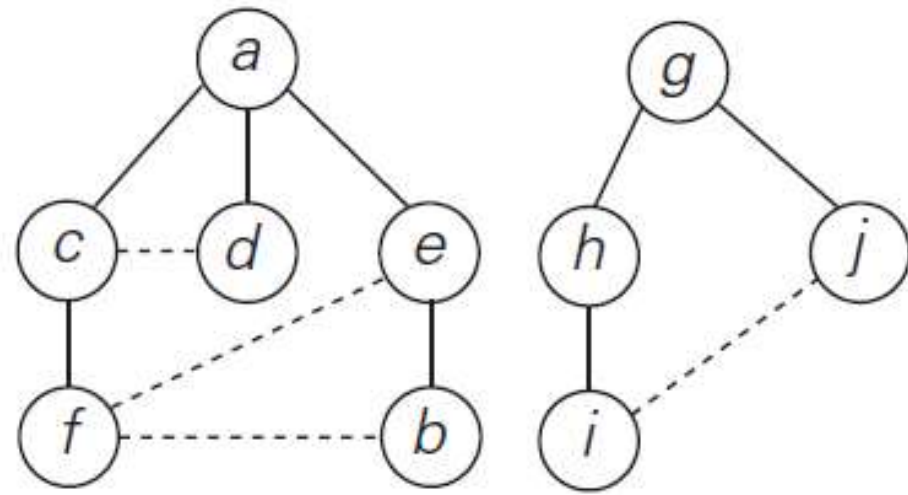
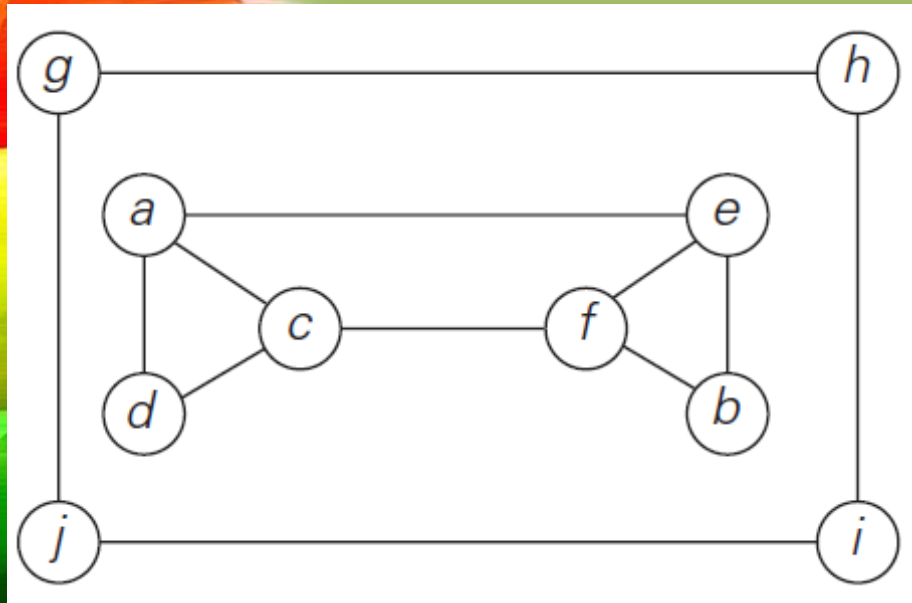


$a_1 c_2 d_3 e_4 f_5 b_6$
 $g_7 h_8 j_9 i_{10}$

(b) Traversal queue



Breadth-First Search(DFS)



(c) BFS forest with the tree and cross edges



Breadth-First Search(BFS)

ALGORITHM $BFS(G)$

//Implements a breadth-first search traversal of a given graph

//Input: Graph $G = \langle V, E \rangle$

//Output: Graph G with its vertices marked with consecutive integers

// in the order they are visited by the BFS traversal

mark each vertex in V with 0 as a mark of being “unvisited”

$count \leftarrow 0$

for each vertex v in V **do**

if v is marked with 0

$bfs(v)$



Breadth-First Search(BFS)

bfs(v)

//visits all the unvisited vertices connected to vertex v

//by a path and numbers them in the order they are visited

//via global variable *count*

$count \leftarrow count + 1$; mark v with *count* and initialize a queue with v

while the queue is not empty **do**

for each vertex w in V adjacent to the front vertex **do**

if w is marked with 0

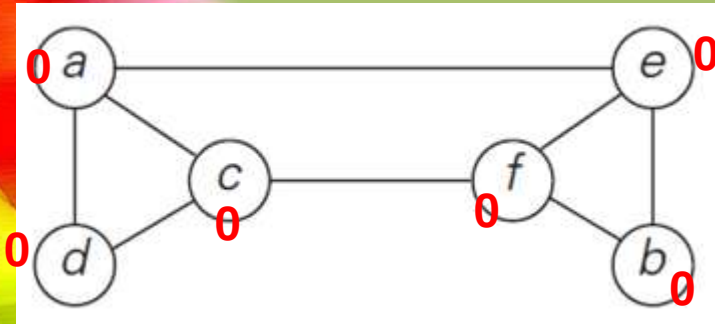
$count \leftarrow count + 1$; mark w with *count*

 add w to the queue

 remove the front vertex from the queue



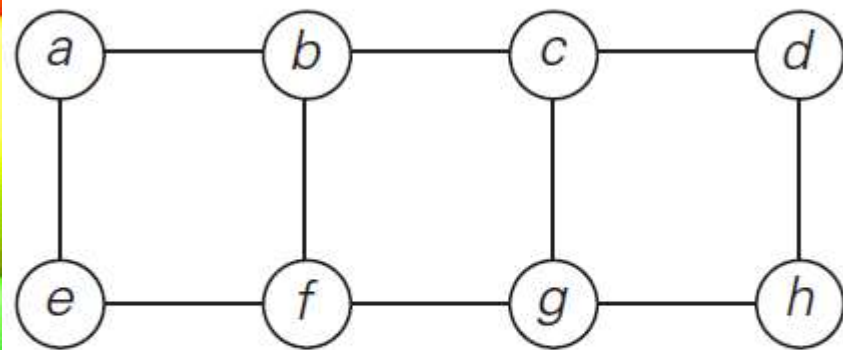
Breadth-First Search(DFS)



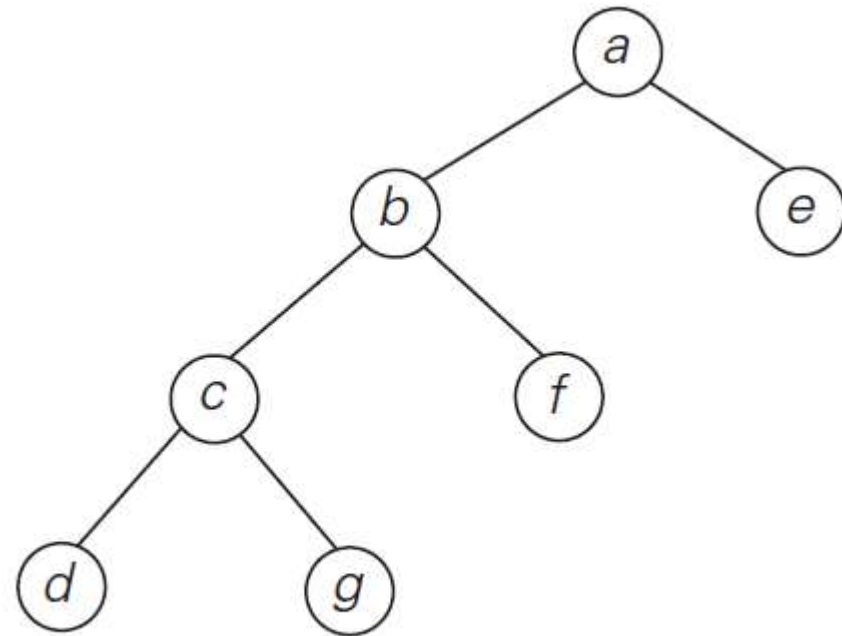
1	2	3	4	5	6
a	c	d	e	f	b



Breadth-First Search(BFS)



(a)



(b)

(a) Graph.

(b) Part of its BFS tree that identifies the minimum-edge path from *a* to *g*



สรุป

	DFS	BFS
Data structure	a stack	a queue
Number of vertex orderings	two orderings	one ordering
Edge types (undirected graphs)	tree and back edges	tree and cross edges
Applications	connectivity, acyclicity, articulation points	connectivity, acyclicity, minimum-edge paths
Efficiency for adjacency matrix	$\Theta(V ^2)$	$\Theta(V ^2)$
Efficiency for adjacency lists	$\Theta(V + E)$	$\Theta(V + E)$



อ้างอิงและต้นฉบับ

[1] Introduction to the design & analysis of algorithm, 3rd,
Anany Levitin